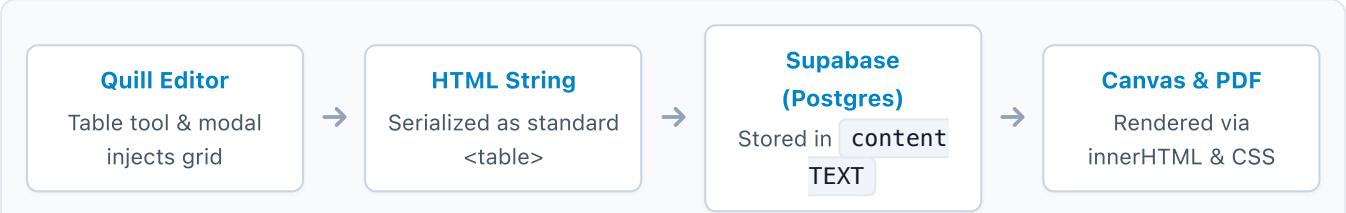
ARCHITECTURE & SYSTEM DESIGN

Table Content Architecture in PaperForm

Complete technical overview of table creation, serialization, database storage, and print rendering.

> **Key Principle:** PaperForm treats tables as **first-class inline HTML entities** rather than separate relational database rows or standalone JSON ASTs. Tables are embedded directly inside rich-text content fields.

1 End-to-End Lifecycle



2 Editor Layer (Quill.js)

The rich-text interface in `Editor.tsx` implements `react-quill` with the official table module enabled:

```
// Module configuration
const modules = useMemo(() => ({
  toolbar: {
    container: `#${toolbarId}`,
    handlers: {
      table: () => setShowTableDialog(true)
    }
  },
  table: true
}), [toolbarId]);

// Modal confirmation handler
const confirmTableInsert = () => {
  const tableModule = quill.getModule('table') as any;
  if (tableModule) {
    quill.insertText(range?.index || quill.getLength() - 1, '\n');
    tableModule.insertTable(tableRows + 1, tableCols);
  }
  setShowTableDialog(false);
};
```

Quill outputs standard semantic HTML elements (`<table>` , `<tbody>` , `<tr>` , `<td>`) directly into its string value.

3 Data Models & Database Schema

Tables reside inside existing text and JSON attributes across the platform:

Context / Entity	Field Name	Database Type	Storage Format
Questions (Stem)	`content`	`TEXT NOT NULL`	Raw HTML string including `<table>...</table>`
Question Options	`options`	`JSONB` (Array)	JSON array of strings: `["Option A", "<table>...</table>"]`
Section Notes	`headerConfig.sectionDescriptions`	`JSONB` (Object)	Key-value map of section names to HTML descriptions
Shared Question Bank	`shared_questions.content`	`TEXT NOT NULL`	HTML markup with deduplication via stripped text hashing

4 Deduplication & Similarity Handling

In the Shared Question Bank (`utils/sharedBank.ts`), questions containing tables are compared using tag-stripped text normalization:

```
export function normalizeText(htmlOrText: string): string {
  return htmlOrText
    .replace(/<[^>]*>/g, ' ')   // Strips <table>, <tr>, <td>, etc.
    .replace(/ /g, ' ')
    .replace(/[^\w\s]/g, '')
    .replace(/\s+/g, ' ')
    .trim()
    .toLowerCase();
}
```

This allows exact deduplication (SHA-256) and fuzzy similarity matching (>95%) based on actual cell content rather than differing HTML formatting attributes.

5 Canvas Rendering & CSS Layout

During preview and layout construction, the raw HTML is rendered into the DOM:

```
<div className="q-text" dangerouslySetInnerHTML={{ __html: (question.content || '').replace(/ /
```

To avoid table collapsing or distortion in both screen view and PDF/print export, `index.css` enforces rigid table-layout rules:

```
.q-text table, .clean-question .q-text table, .ql-editor table {
  table-layout: auto !important;
  border-collapse: collapse;
}
.q-text table td, .q-text table th {
  display: table-cell !important;
  border: 1px solid #000;
  padding: 4px 8px;
}
.q-text table tr {
  display: table-row !important;
}
```

6 PDF & Export Generation

When the user triggers **Export PDF**, the `#printable-paper` DOM tree is captured by the PDF generator (e.g. html2pdf / print engine). Because tables are native DOM elements, cell dimensions, borders, and embedded formulas remain pixel-perfect without requiring any secondary transformation.